



COMSATS University Islamabad (CUI)

Department of Software Engineering

Assignment 2

Advanced Web Technologies

Submitted By:

Hamza Farooq (FA23-BSE-038)

Hamza (FA23-BSE-037)

Submission Date: 08/04/2026

**Bachelor of Science in Software
Engineering (2023- 2027)**

1) Brief Project Overview

The **Personal Finance Analytics Dashboard** is a MERN-based web application that allows users to manage their personal finances through expense tracking, budgeting, savings goals, and analytics.

The system enables users to:

- record income and expenses
- categorize transactions
- monitor monthly budgets
- visualize spending patterns
- set savings goals
- generate reports
- import/export transaction history

The project focuses on:

- secure REST API development
- role-based authentication
- data aggregation
- advanced filtering
- dashboard reporting
- middleware architecture

2) Modules / Entities / Routers Overview

Modules

1. Authentication Module
2. User Profile Module
3. Transactions Module
4. Budget Module
5. Savings Goals Module
6. Reports & Analytics Module
7. Notifications Module

Entities

User

- id
- fullName
- email

- passwordHash
- role
- currencyPreference
- createdAt

Transaction

- id
- userId
- type (income/expense)
- amount
- category
- description
- transactionDate
- paymentMethod

Budget

- id
- userId
- category
- monthlyLimit
- spentAmount
- remainingAmount

SavingsGoal

- id
- userId
- title
- targetAmount
- currentAmount
- deadline
- status

Routers

- /api/auth
- /api/users
- /api/transactions
- /api/budgets
- /api/goals
- /api/reports
- /api/notifications

3) API Endpoints Specification

A) Authentication Module

1. Register User

- **Method:** POST
- **Route:** /api/auth/register
- **Description:** Creates a new user account.

Request Body

```
{  
  "fullName": "Ali Khan",  
  "email": "ali@example.com",  
  "password": "StrongPass123"  
}
```

Response (201)

```
{  
  "message": "User registered successfully",  
  "user": {  
    "id": "u123",  
    "fullName": "Ali Khan",  
    "email": "ali@example.com"  
  }  
}
```

Errors

- 400 → Missing fields
- 409 → Email already exists
- 500 → Server error

Scenarios

Normal Flow

- User submits valid data
- Password is hashed
- User stored in DB

Alternate Flow

- Existing email → return conflict error

Exception

- DB unavailable → 500

2. Login

- **Method:** POST

- **Route:** /api/auth/login

Request Body

```
{  
  "email": "ali@example.com",  
  "password": "StrongPass123"  
}
```

Response (200)

```
{  
  "token": "jwt_token_here",  
  "user": {  
    "id": "u123",  
    "fullName": "Ali Khan"  
  }  
}
```

Errors

- 401 → Invalid credentials
- 404 → User not found

B) Transactions Module

1. Create Transaction

- **Method:** POST
- **Route:** /api/transactions
- **Auth:** Required

Request Body

```
{  
  "type": "expense",  
  "amount": 2500,  
  "category": "Food",  
  "description": "Dinner",  
  "transactionDate": "2026-04-08"  
}
```

Response (201)

```
{  
  "id": "t1001",  
  "type": "expense",  
  "amount": 2500,  
  "category": "Food"  
}
```

Errors

- 400 → Invalid amount
- 401 → Unauthorized
- 422 → Invalid category

Scenarios

Normal

- Authenticated user creates transaction

Alternate

- Missing optional description → saved as null

Exception

- Negative amount → validation error

2. Get All Transactions

- **Method:** GET
- **Route:** /api/transactions
- **Query Params**
 - category
 - startDate
 - endDate
 - type
 - page
 - limit

Response (200)

```
{
  "transactions": [],
  "pagination": {
    "page": 1,
    "totalPages": 5
  }
}
```

3. Update Transaction

- **Method:** PUT
- **Route:** /api/transactions/:id

Errors

- 404 → Transaction not found
- 403 → Unauthorized update attempt

4. Delete Transaction

- **Method:** DELETE
- **Route:** /api/transactions/:id

Response

```
{  
  "message": "Transaction deleted"  
}
```

C) Budget Module

1. Create Monthly Budget

- **Method:** POST
- **Route:** /api/budgets

Request

```
{  
  "category": "Food",  
  "monthlyLimit": 20000  
}
```

Response

```
{  
  "category": "Food",  
  "monthlyLimit": 20000,  
  "spentAmount": 0  
}
```

Exception Cases

- Duplicate budget for same month/category
- Negative limit
- Invalid category

D) Savings Goals Module

1. Create Goal

- **Method:** POST
- **Route:** /api/goals

Request

```
{
  "title": "Laptop",
  "targetAmount": 150000,
  "deadline": "2026-12-01"
}
```

Response

```
{
  "goalId": "g101",
  "status": "active"
}
```

E) Reports & Analytics Module

This is the **main strength** of your project.

1. Monthly Expense Summary

- **Method:** GET
- **Route:** /api/reports/monthly-summary
- **Auth:** Required

Response

```
{
  "month": "April",
  "income": 120000,
  "expenses": 75000,
  "savings": 45000
}
```

Scenario

Uses **MongoDB** aggregation pipeline.

2. Category Wise Spending

- **Method:** GET
- **Route:** /api/reports/category-breakdown

Response

```
{
  "Food": 15000,
  "Transport": 8000,
  "Shopping": 12000
}
```

3. Budget vs Actual

- **Method:** GET
- **Route:** /api/reports/budget-vs-actual

Response

```
{
  "Food": {
    "budget": 20000,
    "spent": 18000
  }
}
```

4) Error Handling Structure

A consistent global error response format will be used.

```
{
  "success": false,
  "message": "Validation failed",
  "errorCode": "INVALID_AMOUNT",
  "details": {}
}
```

HTTP Error Codes

- 400 → Bad Request
- 401 → Unauthorized
- 403 → Forbidden
- 404 → Not Found
- 409 → Conflict
- 422 → Validation Error
- 500 → Internal Server Error

5) Middleware Functionalities

1. Authentication Middleware

Purpose

Verifies JWT token.

Used In

- transactions
- budgets
- reports
- goals

2. Authorization Middleware

Checks if user owns the resource.

Example:

- user can only update own transactions

3. Validation Middleware

Validates:

- request body
- params
- query filters

Using:

- Joi / express-validator

4. Error Handling Middleware

Centralized exception handler for:

- DB errors
- validation errors
- duplicate keys
- token expiry

5. Logging Middleware

Logs:

- request method
- route
- response time
- user id

Useful for debugging viva questions.